

Séance 3 — Exercice : l'écran de gestion des réservations

KFOKAM48 — Batch 2 — Phase 3 • Séances Full Stack
Frontend — Angular ou React, au choix

Le contexte

En séance 2, vous avez construit l'API du module Réservation. Vous lui donnez maintenant une interface.

Vous consommez **votre propre backend**. S'il est incomplet, terminez-le d'abord : un écran branché sur une API qui ne fonctionne pas ne vaut rien.

Aucun code ne vous sera donné. La documentation officielle de votre framework et le projet existant sont vos seules sources.

Ce qu'il faut livrer

Un écran unique, accessible depuis la navigation du projet, comportant :

1. La **liste des réservations**
 2. Un **formulaire de création**
 3. Une **action d'annulation** sur chaque ligne
-

1. La liste

Colonnes affichées : titre du livre, nom de l'adhérent, statut, date de réservation, date d'expiration, action.

Un filtre par statut : tous, EN_ATTENTE, DISPONIBLE, ANNULEE, EXPIREE, HONOREE.

Les quatre états à gérer

C'est le cœur de l'exercice. Un écran qui n'affiche que le cas nominal est incomplet.

État	Ce que l'utilisateur doit voir
Chargement	Un indicateur visible pendant l'appel — jamais un écran figé
Données	Le tableau rempli
Liste vide	Un message explicite : « Aucune réservation », pas un tableau vide sans en-tête
Erreur	Un message compréhensible et un moyen de réessayer

Pour tester l'état d'erreur : arrêtez votre backend et rechargez la page. L'écran doit expliquer que le serveur est injoignable, pas rester bloqué sur le chargement.

2. Le formulaire de création

Deux champs : **livre** et **adhérent**, présentés en listes déroulantes alimentées par vos endpoints existants — pas de saisie d'identifiant à la main.

Validation avant envoi : le bouton reste inactif tant que les deux champs ne sont pas renseignés.

Après un succès : la liste se rafraîchit et la nouvelle réservation apparaît, sans rechargement de la page.

Le traitement des refus métier

C'est ce qui distingue cet exercice d'un formulaire d'école. Votre API renvoie des **409** porteurs de sens, et l'utilisateur doit les comprendre.

Situation	Code reçu	Ce qui doit s'afficher
Livre disponible	409	Le message du serveur, lisible, à côté du formulaire
Réservation déjà existante sur ce livre	409	Idem
Quota de 3 réservations atteint	409	Idem
Champ manquant	400	Le message de validation du serveur
Livre ou adhérent inexistant	404	Un message adapté

Interdit : un `alert()`, un message générique du type « une erreur est survenue », ou pire, un échec silencieux où rien ne se passe quand l'utilisateur clique.

3. L'annulation

Un bouton par ligne, qui appelle `PATCH /api/reservations/{id}/annuler`.

- Il n'apparaît que si le statut est `EN_ATTENTE` ou `DISPONIBLE`
 - Une confirmation est demandée avant l'appel
 - En cas de succès, le statut se met à jour dans la liste
 - En cas de **409**, le message du serveur s'affiche
-

Les exigences d'architecture

Un service dédié aux appels API. Aucun `fetch`, `axios` ou `HttpClient` appelé directement depuis un composant.

Découpage en composants. Au minimum : un conteneur qui gère l'état et les appels, un composant de liste, un composant de formulaire.

Aucune donnée codée en dur. Tout vient de l'API.

Interface en français, cohérente avec le reste du projet.

Barème

Élément	Points
Liste fonctionnelle avec ses colonnes et le filtre par statut	8
Les quatre états correctement gérés	6
Formulaire fonctionnel, listes déroulantes alimentées par l'API	6
Affichage lisible des erreurs métier (409, 400, 404)	8
Annulation avec confirmation et mise à jour de la liste	5
Architecture : service isolé, composants découpés	5
Total	38

Le poste le mieux doté est le traitement des erreurs. C'est délibéré : afficher des données est facile, gérer ce qui rate est ce qui sépare un écran d'école d'un écran livrable.

Le passage devant le formateur

Vous présenterez votre écran en **8 minutes**, en le manipulant vous-même. Préparez votre environnement à l'avance : backend démarré, base contenant un livre disponible et plusieurs livres empruntés.

Attendez-vous à devoir provoquer des erreurs en direct, pas seulement à montrer le cas qui fonctionne.

Livrable

Une branche `feature/reservation-ui-prenom-nom`, une Pull Request décrite, relue par un pair.

Dans la description : **quatre captures d'écran** — état de chargement, liste remplie, liste vide, et un refus **409** affiché.

Si vous finissez en avance

- Pagination de la liste
- Tri par date d'expiration
- Badge de couleur par statut
- Compteur de réservations actives par adhérent, avec alerte à partir de 3